

Mal Unified Payments Platform, Deliverable 2

April 28, 2026

Mal has three product squads (Cards, Transfers, Bill Payments) shipping payment events into three different schemas. Each squad owns its own data in its own database. There is no cross-product event store. The pain shows up in four places:

- D1 was scoped to prove that a single canonical event model can serve all four consumers, that bad rows from one squad do not block the others, and that the implementation can run locally in 30 seconds with no Docker and no orchestrator. D2 defends those choices, lays out how the squads onboard in production, and names the work cut from D1 that we will need at 100K transactions per day.

```

[Squad systems]                                [Platform]                                [Consumers]

Cards      -----> Adapter (Cards)      -----+
           |                               |
Transfers  -----> Adapter (Transfers)  -----+----> Validation (Pydantic)  +----> Canonical event stor
           |                               |                               |      Parquet, Hive partiti
Bill Payments -----> Adapter (Bills)  -----+                               |      (event_date, payment
                                           |
                                           +----> Quarantine
                                           (per squad, per run

Canonical store -----> DuckDB (analyst SQL)

```

```
+---> Streamlit data quality dashbo
+---> Finance reconciliation
+---> CBUAE corridor reports
```

The canonical event has five load-bearing decisions, each a direct response to a real failure mode.

Event-grain, not payment-grain. A card transaction is “auth” plus “capture” plus possibly “refund”. The Cards squad models this as one row with two timestamps; the Bill Payments squad models it as one row with a single status field. Neither approach extends to lifecycle queries across products. The canonical model keeps each event as its own row and links related events via `correlation_id`. The auth-to-settle latency query in D1 (SQL 03, p50/p95/p99 across captures) cannot be written cleanly on the squad-grained data.

Integer minor units. Cards stores `auth_amount_usd` as a float. We have already lost reconciliation accuracy to floating-point error. The canonical schema stores `amount_minor` as an integer in the smallest currency unit, parsed at the squad boundary using `decimal.Decimal` with banker’s rounding (`ROUND_HALF_EVEN`). The v1 to v2 migration in `schema/migrations.py` is the recovery path for already-stored v1 data.

raw_payload preserved per row. Adoption is the hardest part of any platform play. Squads will resist if they think they lose information by handing data to us. Every canonical row carries the full original payload as JSON. The squad does not lose anything; the platform earns trust; auditability is automatic.

payment_method_details as a JSON bag. When the Wallet squad ships in Q3, we add an adapter, not a schema migration. The strict canonical fields cover identity, amount, status, and parties. Anything squad-specific lives in a JSON bag. This bounds the cost of new payment types to one adapter file.

Quarantine, not fail-fast. Validation produces (valid, invalid) batches. Bad rows from Cards do not block the day’s Transfers ingest. Quarantine files carry the original payload plus the Pydantic or `AdapterParseError` reason, and live under `data/output/quarantine/{run_ts}/{squad}.json`. Pipeline exit code stays zero even with quarantined records; non-zero exit is reserved for infrastructure failures.

3. Migration strategy

The canonical model only earns its keep if all three squads land on it. We have three years of squad-specific data, three squad PMs with their own roadmaps, and downstream consumers who are already wired into the squad-specific tables. The migration runs in four phases.

Phase 0, shadow mode (weeks 1 to 4). The adapter for each squad reads from the squad’s existing database (or daily CSV export) and writes to the canonical store. Squad systems remain unchanged. We compare row counts and amounts daily and publish a delta report. Zero blast radius. The phase exists to prove the canonical model handles each squad’s reality, including the messy edges (the “AED twelve fifty” amount strings, the lowercase bill statuses, the Asia/Dubai naive timestamps) that D1’s mock data deliberately reproduces.

Phase 1, dual-write (weeks 5 to 8). The squad writes into the canonical store via the adapter in addition to its own database. Downstream consumers (Finance, Customer Success) start reading from the canonical store. Squad-specific dashboards keep working. Failure-code vocabularies are normalized at this stage; squad PMs sign off on the mappings (DECLINED 51 maps to “Insufficient funds”, and so on).

Phase 2, cutover (weeks 9 to 12). Squad-specific reporting moves to the canonical store. The squad keeps writing to its own transactional database; the canonical store is the analytics source of truth. Old per-squad reports are decommissioned with a 30-day grace window.

Phase 3, schema convergence (weeks 13 to 24). Where the squad PM agrees, the squad’s transactional schema starts moving toward the canonical fields. The amount-as-string in Transfers, the float USD in Cards, and the LOWERCASE statuses in Bill Payments are all candidates. This phase is opt-in and runs at each squad’s pace. The platform does not block on it.

The v1 to v2 schema migration in D1 is the template for any future schema bump. New required field gets a default that makes sense for legacy rows. Banker’s rounding handles numerical conversion. Every output row is re-validated against the new schema before write. Source data is never mutated. The migration is idempotent, so re-running it is safe.

4. Production gaps cut from D1

D1 was scoped for a 30-second local run. At Mal’s expected production volume of around 100K transactions per day across all three squads, the following gaps matter:

Gap	What D1 does	What production needs	Priority
Orchestration	One Python process	Dagster (asset-based model, SQL-native sensors, lineage out of the box)	High
Ingestion	CSV files in <code>data/raw/</code>	Debezium plus Kafka for CDC, with a 30-second freshness SLO	High
Customer identity	<code>customer_id</code> is a string trusted from each squad	Identity service deduplicating across phone, email, and national ID	High
PII tokenization	last4 only, no real PII in mocks	Vault or KMS for tokens; analytics never sees raw PII	High

Gap	What D1 does	What production needs	Priority
Observability	One INFO line per squad per run	Structured logging, per-stage metrics, alerts on quarantine rate spikes	High
Schema registry	Pydantic models in the repo	Confluent Schema Registry or Git-based registry with evolution rules	Medium
Multi-region DR	Single local Parquet directory	UAE-resident primary, second region for DR, CBUAE residency rules	Medium
Backfill and replay	Re-run <code>make seed</code> and <code>make run</code>	Replay-from-Kafka with idempotent writes; backfill jobs in Dagster	Medium
Dashboards	Static GitHub Pages report and Streamlit (D3)	Looker or Superset on the canonical store, with squad-specific saved views	Medium
FX rates	Hardcoded USD, AED, GBP table	Price-service integration with timestamped rates, audit trail	Medium

The High items are non-negotiable for a production launch. The Medium items can land iteratively after the canonical store is live. Critically, none of them change the canonical model. We can ship the canonical store first and add these layers behind it.

Why Dagster, not Airflow

Airflow’s DAG model treats every step as a task. Dagster treats data assets as first-class, with SQL and Python sensors that fire when an upstream asset changes. For a payment platform that has to reconcile across three squads, an asset-based model maps directly to “the canonical event store for date X is fresh”. Airflow can do this with sensors and XComs but the wiring is brittle. Dagster’s lineage UI also gives finance and compliance a way to see where any given canonical row came from, which is a real CBUAE-audit ask.

Why CDC, not nightly extracts

Nightly extracts are fine for monthly finance reporting. They are not fine for fraud monitoring, real-time customer-360, or platform reliability dashboards that are supposed to surface “bill payment

failures spiking right now”. CDC via Debezium gives a 30-second freshness SLO, supports replay (which we will need on day one for backfills), and avoids the daily-batch-window problem that gets worse as squad volume grows.

5. Closing

D1 proved the canonical model and the quarantine pattern. D2 puts a four-phase migration plan and a production-gap punch list against it. D3 (the Streamlit data quality dashboard) gives platform reliability the per-squad ingestion-rate, quarantine-rate, and schema-drift signals it will need to run this in production.

The single biggest risk is adoption, not technology. Phase 0 (shadow mode) is what proves the canonical model handles each squad’s reality without any squad changing anything. Once that delta report is green for two weeks, dual-write and cutover are mechanical. Phase 3 (schema convergence) is the bonus, not the goal. The goal is one canonical event store that Finance, Customer Success, Compliance, and Platform reliability can all rely on.